

Suy nghĩ từ nhiều góc độ, tư duy sáng tạo

Chen Yuxi
Trường Ngoại ngữ Nam Kinh

2007

Nguồn gốc: Thinking from multiple angles and creative thinking: ideas and methods for solving problems with tree dynamic programming

IOI China candidate team papers / 2007 / day 2 / Chen Yuxi

Tóm tắt

Trong vài năm gần đây, các bài toán cần dùng quy hoạch động trên cây xuất hiện thường xuyên trong thi tin học. Những bài toán này có nhiều biến thể, lồng ghép nhiều ý tưởng tinh tế, và đòi hỏi cao ở năng lực phân tích cũng như tư duy sáng tạo của thí sinh. Vì vậy chúng giữ một vị trí quan trọng trong các kỳ thi.

Bài viết phân tích một số bài toán quy hoạch động trên cây trong các kỳ thi quốc tế và quốc gia gần đây, tập trung vào vài cách giải tiêu biểu. Từ quá trình phân tích ấy, ta rút ra phương pháp tư duy chung: nhìn bài toán từ nhiều góc độ và sáng tạo trong cách đặt trạng thái. Mục tiêu không chỉ là nêu cách giải, mà còn trình bày quá trình suy nghĩ dẫn tới cách giải.

Từ khóa. Cấu trúc cây; quy hoạch động; Olympic tin học.

1 Dẫn nhập

Trong thi tin học thường gặp các bài toán kiểu sau: cho một cây, yêu cầu hoàn thành một thao tác nào đó với chi phí nhỏ nhất, hoặc thu được lợi ích lớn nhất. Rất nhiều bài toán được xây dựng bằng cách mở rộng hoặc tăng cường hai yếu tố cơ bản là cấu trúc cây và tính tối ưu, vì vậy trở nên khá khó xử lý.

Những bài toán này thường có kích thước lớn. Thuật toán liệt kê không đủ hiệu quả, còn tham lam thường không bảo đảm tối ưu, nên ta phải dùng quy hoạch động.

Giống quy hoạch động nói chung, khi giải bài toán dạng này ta thường xét ba bước.

1. **Xác lập trạng thái.** Hầu như bài nào cũng cần lưu thông tin của cây con gốc tại một đỉnh nào đó. Tuy nhiên, có cần thêm chiều hay không, thêm bao nhiêu chiều, và thêm theo cách nào thì phải dựa vào từng bài cụ thể.
2. **Chuyển trạng thái.** Cách chuyển rất đa dạng. Đây cũng là trọng tâm phân tích trong các ví dụ của bài viết.
3. **Cài đặt thuật toán.** Nhờ cấu trúc cây, tìm kiếm có nhớ thường là cách cài đặt tự nhiên.

Vì mô hình được xây dựng trên cây, ta gọi đó là quy hoạch động trên cây. Theo nghĩa trực tiếp, đây là quy hoạch động trên cấu trúc dữ liệu “cây”.

Phần lớn các bài quy hoạch động quen thuộc có cấu trúc tuyến tính. Trên đường thẳng có hai hướng tự nhiên: tiến và lùi, tương ứng với tính xuôi và tính ngược. Trên cây cũng có hai hướng tương tự:

1. **Từ gốc xuống lá.** Gốc truyền thông tin hữu ích xuống các con; sau đó nghiệm tối ưu được suy ra từ thông tin đã truyền.
2. **Từ lá lên gốc.** Các con truyền thông tin hữu ích lên cha; sau đó gốc nhận được nghiệm tối ưu. Đây là dạng thường gặp hơn và có phạm vi ứng dụng rộng.

Dĩ nhiên cũng có những bài cần đồng thời cả hai hướng duyệt. Ví dụ đầu tiên dưới đây thuộc loại đó.

2 Ví dụ 1: Tìm Chris

2.1 Mô tả bài toán

Điện thoại nhà Chris reo lên. Từ đầu dây bên kia vang lên giọng hốt hoảng của giáo viên: “A lô, có phải phụ huynh của Chris không? Con anh chị lại không đến lớp, không định tham gia kỳ thi sao?” Nghe đến chuyện thi, bố mẹ Chris rất lo và quyết định tìm Chris trong thời gian ngắn nhất. Họ nói với giáo viên: “Theo kinh nghiệm trước đây, bây giờ Chris chắc chắn đang trốn ở nhà bạn Shermie hoặc Yashiro để chơi *The King of Fighters*. Chúng tôi sẽ đi từ nhà để tìm Chris; hề tìm thấy sẽ gọi lại ngay.” Nói xong họ cúp máy.

Thành phố nơi Chris sống gồm N điểm dân cư và một số đường hai chiều nối giữa chúng. Đi qua đường x mất T_x phút. Giữa hai điểm dân cư bất kỳ có đúng một đường đi, tức mạng đường là một cây. Nhà Chris ở điểm C , Shermie ở điểm A , còn Yashiro ở điểm B . Giáo viên của Chris và bố mẹ Chris đều có bản đồ thành phố; bố mẹ biết vị trí cụ thể của A, B, C , còn giáo viên thì không.

Để nhanh chóng tìm Chris, bố mẹ cậu tuân theo hai quy tắc:

- Nếu A gần C hơn B gần C , họ đến nhà Shermie trước; nếu không thấy Chris, họ đến nhà Yashiro. Trường hợp ngược lại xử lý tương tự.
- Họ luôn đi theo đường đi duy nhất giữa hai điểm.

Giáo viên biết hai quy tắc trên, nhưng không biết vị trí cụ thể của A, B, C . Hãy cho biết trong trường hợp xấu nhất, bố mẹ Chris phải mất bao lâu để tìm được Chris.

Ví dụ, xét cây bốn đỉnh sau, mọi cạnh có độ dài 1.

```
A -- C -- B
    |
    Chris
```

Nếu Chris sống ở C , Shermie ở A , Yashiro ở B , và B gần C hơn A , bố mẹ Chris sẽ đến nhà Yashiro trước. Nếu không thấy Chris ở đó, họ đi tiếp đến nhà Shermie. Khi đó trường hợp xấu nhất mất 4 phút.

Dữ liệu vào và ra. Tệp `hookey.in` có dòng đầu gồm hai số nguyên N và M ($3 \leq N \leq 200000$), lần lượt là số điểm dân cư và số đường. Mỗi dòng trong M dòng tiếp theo chứa U_i, V_i, T_i ($1 \leq U_i, V_i \leq N, 1 \leq T_i \leq 10^9$), nghĩa là đường i nối U_i với V_i và đi qua nó mất T_i phút. Mỗi đường được cho đúng một lần. Tệp `hookey.out` chứa một số nguyên T , là thời gian lớn nhất cần thiết trong trường hợp xấu nhất.

2.2 Phân tích

Trừu tượng hóa. Bài toán tương đương với việc chọn ba đỉnh X, Y, Z trên một cây có trọng số sao cho

$$\text{dist}(X, Y) \leq \text{dist}(X, Z),$$

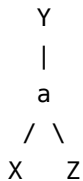
và giá trị

$$\text{dist}(X, Y) + \text{dist}(Y, Z)$$

lớn nhất có thể. Ở đây X là nhà Chris, Y là nơi được đến trước, còn Z là nơi được đến sau.

Phân tích ban đầu. Mô hình là một cây và kích thước rất lớn, nên ta dễ nghĩ đến quy hoạch động trên cây. Nếu cố định một đỉnh Y làm nơi đến trước, ta chỉ cần tìm hai đỉnh xa nhất liên quan tới Y . Nhưng trên cây, tìm hai đỉnh xa nhất từ một đỉnh bất kỳ tốn $\mathcal{O}(n)$; vì chưa biết đỉnh nào là Y , nếu liệt kê Y thì tổng thời gian thành $\mathcal{O}(n^2)$, không thể chạy với $N = 200000$.

Không liệt kê trực tiếp Y không có nghĩa là phải bỏ hẳn ý tưởng liệt kê. Ta thử nhìn bài toán từ một góc khác. Xét dạng đường đi sau:



Hai đường đi $Y \rightarrow X$ và $Y \rightarrow Z$ cùng đi từ Y tới một đỉnh a rồi tách ra. Ta gọi a là **điểm phân nhánh**. Nếu liệt kê điểm phân nhánh, liệu có thể có thuật toán hiệu quả hơn không?

Liệt kê điểm phân nhánh. Giả sử chọn một đỉnh a làm điểm phân nhánh và gốc hóa cây tại a . Đỉnh Y có hai khả năng: $Y = a$, hoặc Y nằm trong một cây con của một con của a . Trường hợp $Y = a$ có thể xem như trường hợp thứ hai bằng cách thêm cho a một con rỗng có cạnh dài 0.

Vì a là điểm phân nhánh, ba đỉnh X, Y, Z phải nằm ở ba nhánh khác nhau đi ra từ a . Khi đó

$$\begin{aligned} \text{dist}(X, Y) + \text{dist}(Y, Z) &= (\text{dist}(X, a) + \text{dist}(Y, a)) + (\text{dist}(Y, a) + \text{dist}(Z, a)) \\ &= \text{dist}(X, a) + 2 \text{dist}(Y, a) + \text{dist}(Z, a). \end{aligned}$$

Điều kiện $\text{dist}(X, Y) \leq \text{dist}(X, Z)$ tương đương $\text{dist}(Y, a) \leq \text{dist}(Z, a)$. Vì vậy, nếu biết ba khoảng cách lớn nhất đi từ a vào ba nhánh khác nhau và sắp chúng thành $d_1 \geq d_2 \geq d_3$, giá trị tốt nhất tại a là

$$d_1 + 2d_2 + d_3.$$

Nếu với mỗi a ta lại quét toàn cây để tìm ba giá trị này, thuật toán vẫn là $\mathcal{O}(n^2)$. Ta cần một cách truyền thông tin tốt hơn.

Hai lần duyệt. Gốc hóa cây tại đỉnh 1. Lần duyệt thứ nhất đi từ lá lên gốc để tính, với mỗi đỉnh, các khoảng cách xa nhất nằm trong những cây con của nó. Lần duyệt thứ hai đi từ gốc xuống lá, chẳng hạn theo BFS để cha luôn được xử lý trước con. Khi đang xét đỉnh a , một điểm xa nhất từ a hoặc nằm trong cây con của a , hoặc nằm ngoài cây con đó. Nếu nằm ngoài, đường đi chắc chắn phải qua cha của a .

Do đó tại mỗi đỉnh ta lưu hai giá trị lớn nhất được truyền tới nó, đồng thời lưu mỗi giá trị đến từ đỉnh nào. Khi truyền thông tin từ một đỉnh sang một con v , nếu giá trị lớn nhất của đỉnh hiện tại đến từ

chính v thì dùng giá trị lớn thứ hai; nếu không thì dùng giá trị lớn nhất. Cộng thêm độ dài cạnh giữa hai đỉnh để được khoảng cách nhìn từ v .

Sau hai lần duyệt, với mỗi đỉnh ta đã có các ứng viên xa nhất ở những nhánh khác nhau. Lấy ba giá trị lớn nhất thuộc ba nguồn kẻ khác nhau, sắp giảm dần thành d_1, d_2, d_3 , rồi cập nhật đáp án bằng $d_1 + 2d_2 + d_3$. Cả thời gian và bộ nhớ đều là $\mathcal{O}(n)$.

Cài đặt. Các giá trị trung gian và đáp án có thể vượt phạm vi số nguyên 32 bit, vì vậy cần dùng kiểu 64 bit.

Ý tưởng trong ví dụ này có tính sáng tạo: thay vì nhìn vào đỉnh bắt đầu hay đỉnh đến trước, ta nhìn vào điểm phân nhánh của hai đường đi. Phương pháp hai lần duyệt và kỹ thuật “lưu vài giá trị lớn nhất cùng nguồn truyền” xuất hiện trong rất nhiều bài toán trên cây.

3 Ví dụ 2: Nhà máy gỗ ở Byteland

3.1 Mô tả bài toán

Gần như toàn bộ vương quốc Byteland được bao phủ bởi rừng và sông. Các sông nhỏ hợp lại thành sông lớn hơn; cuối cùng mọi dòng nước đều đổ vào một con sông lớn, rồi con sông này chảy ra biển. Tại cửa sông có một ngôi làng tên là Bytetown.

Trong Byteland có n làng khai thác gỗ, đều nằm ven sông. Hiện tại ở Bytetown có một nhà máy gỗ lớn xử lý toàn bộ gỗ của cả nước. Sau khi được chặt, gỗ trôi theo sông về nhà máy ở Bytetown. Nhà vua quyết định xây thêm k nhà máy gỗ ở các làng khác để giảm chi phí vận chuyển. Khi đó gỗ sẽ được xử lý tại nhà máy mới đầu tiên mà nó gặp trên đường trôi xuống hạ lưu. Nếu nhà máy đặt ngay tại làng sinh ra gỗ thì gỗ không tốn chi phí vận chuyển.

Tất cả các dòng sông đều không phân nhánh theo chiều xuôi dòng: từ mỗi làng, đi theo hạ lưu chỉ có một đường đến Bytetown. Mỗi làng có sản lượng gỗ hằng năm đã biết. Chi phí vận chuyển là 1 xu cho mỗi khúc gỗ trên mỗi kilomet. Hãy chọn nơi xây k nhà máy mới để tổng chi phí nhỏ nhất.

Dữ liệu vào và ra. Dòng đầu gồm n và k ($2 \leq n \leq 100$, $1 \leq k \leq 50$, $k \leq n$). Các làng ngoài Bytetown được đánh số $1, 2, \dots, n$, còn Bytetown là 0. Trong n dòng tiếp theo, dòng của làng i gồm:

$$w_i, \quad v_i, \quad d_i,$$

trong đó w_i là số khúc gỗ làng i sinh ra mỗi năm ($0 \leq w_i \leq 10000$), v_i là làng gần nhất ở hạ lưu của i ($0 \leq v_i \leq n$), và d_i là khoảng cách từ v_i đến i ($1 \leq d_i \leq 10000$). Kết quả là chi phí vận chuyển nhỏ nhất. Tổng chi phí nếu đưa toàn bộ gỗ về Bytetown không vượt quá 2 000 000 000 xu.

3.2 Phân tích

Trừu tượng hóa. Ta có một cây có gốc là Bytetown. Cần đặt k nhà máy ở các đỉnh khác gốc, sao cho mỗi làng đưa gỗ tới nhà máy tổ tiên gần nhất và tổng chi phí nhỏ nhất. Đây là một bài toán quy hoạch động trên cây khá tự nhiên.

Xác lập trạng thái. Trước hết trạng thái phải biết đỉnh hiện tại và số nhà máy được đặt trong cây con gốc tại đỉnh đó. Nhưng như vậy chưa đủ, vì chi phí còn phụ thuộc vào vị trí nhà máy tổ tiên gần nhất ở phía trên. Ta thêm một chiều biểu diễn tổ tiên đang được dùng làm nơi nhận gỗ nếu trong phần đang xét chưa gặp nhà máy mới.

Đặt

$$f(u, t, q)$$

là chi phí nhỏ nhất trong cây con gốc tại u , khi đặt q nhà máy trong cây con này, và t là một tổ tiên của u đã có nhà máy. Ở đây t chỉ cần là một nhà máy tổ tiên đang được truyền xuống, không nhất thiết phải mô tả mọi chi tiết của đường đi phía trên. Chính việc nói rộng cách hiểu trạng thái này làm chuyển trạng thái đơn giản hơn.

Chuyển trạng thái. Xét hai trường hợp.

Đặt nhà máy tại u . Khi đó gốc của u không tốn chi phí vận chuyển, và các con của u xem u là nhà máy tổ tiên. Một nhà máy đã được dùng tại u , nên số nhà máy còn lại được phân phối cho các cây con. Quá trình phân phối này giống bài toán ba lô trên các con:

$$h_1(i, s) = \min_{0 \leq r \leq s} \{h_1(i-1, s-r) + f(\text{son}_i, u, r)\}.$$

Không đặt nhà máy tại u . Khi đó gốc của u phải đi tới t , tạo chi phí $w_u \text{dist}(u, t)$. Các con vẫn dùng t làm nhà máy tổ tiên đang được truyền xuống:

$$h_2(i, s) = \min_{0 \leq r \leq s} \{h_2(i-1, s-r) + f(\text{son}_i, t, r)\}.$$

Vì vậy, nếu u có c con,

$$f(u, t, q) = \min \{h_1(c, q-1), w_u \text{dist}(u, t) + h_2(c, q)\}.$$

Các mảng h_1, h_2 là mảng tạm để gộp dần các con. Trường hợp cơ sở được xử lý theo số con bằng 0 và số nhà máy còn lại hợp lệ.

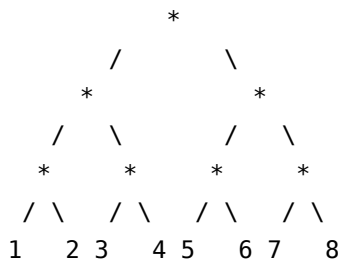
Hiệu quả. Trạng thái có ba chiều. Trong chuyển trạng thái ta phân phối số nhà máy giữa các con, tương tự ba lô. Với $n \leq 100$ và $k \leq 50$, cách cài đặt chuẩn có thể đạt cỡ $\mathcal{O}(n^3 k)$, đủ cho ràng buộc. Điểm đáng chú ý của bài là phải thêm chiều cho vị trí nhà máy tổ tiên, và trong phạm vi các anh em lại thực hiện một quy hoạch động thứ hai kiểu ba lô.

4 Ví dụ 3: Tính phí mạng theo cặp

4.1 Mô tả bài toán

Mạng đã trở thành một phần không thể thiếu của thế giới hiện nay. Mỗi ngày có hàng trăm triệu người dùng mạng để học tập, nghiên cứu, giải trí. Tuy nhiên, bản thân mạng có chi phí vận hành rất lớn; vì vậy thu phí người dùng ở mức thích hợp là cần thiết và hợp lý.

Trường NS ở thành phố MY có một mạng giáo dục như sau. Mạng có 2^N người dùng, đánh số $1, 2, \dots, 2^N$. Người dùng, điểm định tuyến và dây mạng tạo thành một cây nhị phân đầy đủ. Mỗi lá là một người dùng, mỗi đỉnh trong là một điểm định tuyến, mỗi cạnh là một dây mạng. Ví dụ với $N = 3$:



Công ty mạng MY có cách thu phí đặc biệt gọi là “thu phí theo cặp”. Với mỗi cặp người dùng i, j ($1 \leq i < j \leq 2^N$), công ty tính một khoản phí. Mỗi người dùng tự chọn một trong hai phương thức trả phí A hoặc B. Tổng phí mà trường phải trả là tổng đóng góp của mọi cặp.

Với hai người dùng i, j , gọi P là tổ tiên chung gần nhất của họ. Trong các lá thuộc quyền quản lý của P , gọi n_A, n_B lần lượt là số người chọn A và B. Hệ số tính phí k cho cặp i, j được cho bởi bảng sau; $F_{i,j}$ là lưu lượng giữa hai người dùng này.

Phương thức của i	Phương thức của j	Điều kiện	Hệ số k
A	A	$n_A < n_B$	2
A	B	$n_A < n_B$	1
B	A	$n_A < n_B$	1
B	B	$n_A < n_B$	0
A	A	$n_A \geq n_B$	0
A	B	$n_A \geq n_B$	1
B	A	$n_A \geq n_B$	1
B	B	$n_A \geq n_B$	2

Phí thực tế của cặp là $kF_{i,j}$. Vì tổng phí phụ thuộc vào lựa chọn phương thức, học sinh muốn thay đổi phương thức để giảm phí. Nhưng công ty đã lưu phương thức đăng ký ban đầu; nếu người dùng i muốn đổi từ A sang B hoặc ngược lại thì phải trả thêm C_i để sửa hồ sơ. Hãy tìm tổng chi phí nhỏ nhất, gồm phí sửa hồ sơ và phí theo cặp.

Dữ liệu vào và ra. Dòng đầu là số nguyên dương N . Dòng thứ hai có 2^N số, mô tả phương thức đăng ký ban đầu của từng người dùng; 0 nghĩa là A, 1 nghĩa là B. Dòng thứ ba có 2^N số $C_1, C_2, \dots, C_{2^N}$. Các dòng còn lại cho bảng lưu lượng F : ở dòng tương ứng với i , cột j cho $F_{i,i+j}$. Cần in ra tổng chi phí nhỏ nhất.

Ràng buộc: 40% dữ liệu có $N \leq 4$, 80% dữ liệu có $N \leq 7$, toàn bộ dữ liệu có $N \leq 10$, $0 \leq F_{i,j} \leq 500$, $0 \leq C_i \leq 500000$.

4.2 Phân tích

Biến đổi bài toán. Ràng buộc gợi ý quy hoạch động trên cây, nhưng quy tắc thu phí theo cặp làm trạng thái ban đầu khó đặt. Quan sát bảng hệ số, ta thấy nó có dạng rất đặc biệt:

$$2, 1, 1, 0 \quad \text{và} \quad 0, 1, 1, 2.$$

Với hai người dùng i, j , gọi LCA của họ là p . Khi $n_A < n_B$ tại p , mỗi đầu mút chọn A đóng một lần $F_{i,j}$. Khi $n_A \geq n_B$, mỗi đầu mút chọn B đóng một lần $F_{i,j}$. Nhờ vậy, phí theo cặp có thể được chuyển thành phí gán cho từng người dùng trong cặp, tổng cuối cùng không đổi. Đây là tiền đề để làm quy hoạch động.

Xác lập trạng thái. Trạng thái cần biết trong miền của một đỉnh i có bao nhiêu người dùng chọn A. Nhưng như vậy vẫn chưa đủ, vì một người dùng còn bị thu phí bởi các cặp có LCA ở các tổ tiên của i ; tại mỗi tổ tiên ấy, việc thu theo A hay B phụ thuộc vào so sánh n_A, n_B .

Đặt

$$f(i, j, k)$$

là chi phí nhỏ nhất trong miền lá do đỉnh i quản lý, khi có đúng j người chọn A. Với mỗi tổ tiên u của i , ghi nhãn 0 nếu tại u có $n_A < n_B$, và nhãn 1 nếu $n_A \geq n_B$. Dãy nhãn của các tổ tiên được mã hóa bằng số nhị phân k . Trạng thái $f(i, j, k)$ lưu chi phí nhỏ nhất dưới điều kiện đó.

Chuyển trạng thái. Giả sử i có con trái L và con phải R . Ta phân phối j người chọn A giữa hai con. Nếu con trái có u người chọn A , con phải có $j - u$. Từ j và kích thước miền của i , ta biết nhân mới s của đỉnh i :

$$s = \begin{cases} 0, & j < \text{size}(i) - j, \\ 1, & j \geq \text{size}(i) - j. \end{cases}$$

Nhân này được thêm vào dãy tổ tiên khi đi xuống hai con, tức mã mới là $\text{push}(k, s)$. Khi đó

$$f(i, j, k) = \min_u \{f(L, u, \text{push}(k, s)) + f(R, j - u, \text{push}(k, s))\}.$$

Các khoản phí phát sinh giữa hai miền con được tính ở mức i , vì LCA của mọi cặp một lá bên trái và một lá bên phải chính là i . Cách viết trên gom phần ấy vào chi phí con hoặc cộng thêm khi hợp nhất, tùy cách cài đặt.

Biên. Khi i là lá, từ dãy nhân k ta biết người dùng ở lá này phải đóng phí cho các cặp qua những tổ tiên nào theo phương thức A hay B . Cộng thêm chi phí đối phương thức nếu phương thức cuối cùng khác phương thức đăng ký ban đầu.

Hiệu quả. Đặt $m = 2^N$. Nhìn thô, không gian có vẻ là $\mathcal{O}(m^3)$ và thời gian là $\mathcal{O}(m^4)$, vượt xa giới hạn. Tuy nhiên rất nhiều trạng thái không thể xảy ra.

Xem gốc là tầng 0, lá là tầng N . Ở tầng i , mỗi đỉnh quản lý nhiều nhất 2^{N-i} lá, nên số giá trị khả dĩ của j chỉ là $2^{N-i} + 1$. Số tổ tiên chỉ là i , nên số mã k nhiều nhất là 2^i . Tích hai chiều sau của mỗi đỉnh là $\mathcal{O}(2^{N-i} 2^i) = \mathcal{O}(m)$. Nếu tổ chức bộ nhớ theo tầng hoặc theo tìm kiếm có nhớ, không gian thực tế là $\mathcal{O}(m^2)$.

Về thời gian, tại tầng i có 2^i đỉnh. Mỗi đỉnh có $\mathcal{O}(m)$ trạng thái, và mỗi chuyển trạng thái phân phối số lượng A trong phạm vi $\mathcal{O}(2^{N-i})$. Tổng thời gian mỗi tầng là $\mathcal{O}(m^2)$, nên toàn bộ thời gian là

$$\mathcal{O}(m^2 N).$$

Cài đặt nên dùng tìm kiếm có nhớ và thao tác bit để xử lý mã k .

Ngoài thiết kế quy hoạch động, bài này có hai điểm khó: biến đổi quy tắc thu phí, và phân tích lại độ phức tạp bằng cách đếm trạng thái khả dĩ. Trong các bài trên cây, nhiều thuật toán nhìn qua có vẻ quá chậm, nhưng sau khi phân tích phân bổ theo tầng hoặc theo kích thước cây con, độ phức tạp thực tế có thể thấp hơn một chiều hoặc hơn nữa.

5 Ví dụ 4: Hang động và trò đoán kho báu

5.1 Mô tả bài toán

Nước S có một hang động gồm n phòng và một số hành lang. Giữa hai phòng bất kỳ luôn có đúng một đường đi. A giấu nhiều kho báu trong một phòng nhưng không nói cho B biết phòng nào. B muốn tìm vị trí kho báu nên được đoán nhiều lần; mỗi lần B đoán một phòng có kho báu. Nếu đoán đúng, A nói rằng B đã đúng. Nếu đoán sai, A cho biết từ phòng B vừa đoán phải đi qua hành lang nào đầu tiên để tới kho báu; A chỉ nói hành lang đầu tiên đó.

Hãy đọc mô tả hang động từ `jas.in`, tính số lần đoán ít nhất cần thiết trong trường hợp xấu nhất để B biết vị trí kho báu, rồi ghi kết quả ra `jas.out`.

Dữ liệu vào và ra. Dòng đầu có n ($1 \leq n \leq 50000$). Các phòng đánh số từ 1 đến n . Trong $n - 1$ dòng tiếp theo, mỗi dòng chứa hai số a, b ($1 \leq a, b \leq n$), nghĩa là có một hành lang giữa a và b . Chương trình in ra một số nguyên: số lần đoán ít nhất cần thiết trong trường hợp xấu nhất.

Ví dụ.

```
jas.in
5
1 2
2 3
4 3
5 3
```

```
jas.out
2
```

5.2 Phân tích

Trước hết, ta dễ đoán rằng bài toán liên quan tới quy hoạch động trên cây. Nhưng nếu chỉ đặt trạng thái là “với cây con gốc tại i , cần ít nhất bao nhiêu lần hỏi” thì chưa đủ.

Xác lập trạng thái. Sau mỗi lần hỏi, B được biết kho báu nằm trong một thành phần liên thông nào đó. Nếu thành phần còn lại không chứa đỉnh i , thì số lần hỏi đã dự tính cho cây con gốc i là đủ. Ngược lại, nếu thành phần vẫn chứa i , việc hỏi có thể đã lãng phí một phần thông tin: kho báu còn nằm trong cây con, nhưng cấu hình chưa chắc giống như sau khi giảm đúng một lần hỏi.

Vì vậy ta cần một dãy tham số. Với cây con gốc tại i , đặt

$$Z_{i,0}, Z_{i,1}, Z_{i,2}, \dots$$

để mô tả các mức “dư địa” sau những lần hỏi mà thành phần còn lại vẫn chứa i . Trực giác là: nếu sau lần hỏi đầu tiên ta vẫn biết kho báu ở trong cây con và thành phần chứa i , thì cần ít nhất $Z_{i,1}$ lần hỏi nữa trong tình huống xấu nhất; sau lần thứ hai tương tự là $Z_{i,2}$, v.v. Khi chọn chiến lược, ta muốn $Z_{i,0}$ nhỏ nhất; nếu hòa thì muốn $Z_{i,1}$ nhỏ nhất; tiếp tục như vậy.

Điểm đột phá là không chỉ hỏi “cây con này cần bao nhiêu lần”, mà còn ghi lại nếu các lần hỏi trước vẫn chưa loại được phía chứa gốc thì ta còn cần bao nhiêu. Có những trường hợp hai phương án cùng cần số lần hỏi tối đa như nhau, nhưng phương án có dãy Z tốt hơn sẽ hữu ích hơn khi ghép với các cây con khác.

Hai ví dụ chuyển trạng thái. Xét một đỉnh A có hai con i và j .

Ví dụ thứ nhất:

$$Z_i = (5, 3, 1), \quad Z_j = (4, 2).$$

Ta có thể hỏi trước trong cây con i . Nếu kho báu nằm trong i , 5 lần hỏi là đủ. Nếu không, chuyển sang cây con j ; nếu kho báu ở đó, cần 4 lần nữa, cộng với một lần trước là 5. Nếu vẫn không rơi vào nhánh đó, quay lại xét mức tiếp theo của i , cần 3 lần nữa cộng hai lần trước, cũng là 5. Vì vậy tổng cộng 5 lần là đủ mà không cần hỏi ngay tại A .

Ví dụ thứ hai:

$$Z_i = (4, 2, 1), \quad Z_j = (2).$$

Nếu hỏi trong cây con i trước và kho báu nằm trong đó, cần 4 lần. Nếu không, ta hỏi một lần tại A , khi đó biết chắc kho báu ở nhánh i hay nhánh j . Dù ở nhánh nào, số lần còn lại đều là 2, nên tổng cộng cũng là 4. Ở đây có lúc phải hỏi tại A , nhưng không nhất thiết hỏi A ngay từ đầu.

Cài đặt thuật toán. Với mỗi đỉnh i , chương trình lưu một bảng Z_i có độ dài $\lceil \log_2 n \rceil + O(1)$; có thể chứng minh số lần hỏi cỡ logarit là đủ. Với lá, hiển nhiên

$$Z_{i,0} = 1,$$

các vị trí còn lại bằng 0.

Với đỉnh hiện tại A , trước hết cộng các dãy Z của mọi con theo từng vị trí:

$$Z_A = \sum_{\text{con } v \text{ của } A} Z_v.$$

Nếu một vị trí của Z_A có giá trị 1, ta hiểu rằng có đúng một cây con cần được hỏi ở mức đó; ta có thể hỏi trong cây con ấy theo cách của ví dụ thứ nhất.

Gọi x là vị trí lớn nhất sao cho $Z_{A,x} > 0$. Ta cần tìm vị trí nhỏ nhất $y > x$ với $Z_{A,y} = 0$. Khi đó hỏi một lần tại A ở mức y , đặt

$$Z_{A,y} = 1, \quad Z_{A,0} = Z_{A,1} = \dots = Z_{A,y-1} = 0.$$

Nếu lần hỏi tại A trả lời rằng kho báu nằm trên đường đi về cha, thì chắc chắn kho báu không nằm trong cây con gốc A , nên không cần các mức thấp hơn nữa. Đây chính là dãy Z của đỉnh A .

Vì sao khi hòa phải chọn dãy Z nhỏ hơn theo thứ tự từ thấp lên cao? Hãy xem dãy Z như một số nhị phân, trong đó Z_0 là bit thấp nhất. Khi đó $(5, 2)$ tương ứng với 100100_2 , còn $(4, 3)$ tương ứng với 011000_2 ; dãy sau nhỏ hơn. Nếu một dãy lớn hơn vừa khớp lệch với dãy của cây con khác và tạo thành nhiều bit 1, về sau có thể buộc phải hỏi ở gốc sớm hơn. Chọn dãy nhỏ nhất ngay từ đầu cho kết quả tốt hơn.

Bài này khó hơn ba ví dụ trước vì trạng thái không hiện ra trực tiếp. Ta phải bắt đầu từ các ví dụ nhỏ, cảm nhận điều gì đang bị lãng phí sau mỗi lần hỏi, rồi mới tìm được dãy trạng thái phù hợp.

6 Tổng kết

Bốn ví dụ trên minh họa nhiều phương pháp giải bài toán quy hoạch động trên cây: duyệt cây hai lần; ý tưởng điểm phân nhánh; lưu vài giá trị tốt nhất và nguồn truyền của chúng; quy hoạch động nhiều chiều; làm thêm một quy hoạch động kiểu ba lô giữa các con; và phân tích độ phức tạp theo cách phân bố trên cấu trúc cây. Những kỹ thuật này đều có phạm vi ứng dụng rộng trong thi đấu.

Qua các ví dụ, ta thấy khi gặp một bài toán lạ, một mặt cần phân tích sâu các thuộc tính của bài toán để tìm bản chất, mặt khác cần nhìn từ nhiều góc độ và nắm lấy tính chất đặc biệt của nó. Đó là con đường để tạo ra cách giải đúng.

Tất nhiên, các bài toán kiểu này có rất nhiều biến thể. Bài viết chỉ đưa ra một số phương pháp và hướng suy nghĩ; điều quan trọng vẫn là tích lũy và tổng kết thường xuyên trong quá trình luyện tập.

References

- [1] Wu Wenhui, Wang Jiande. *Analysis and Program Design of Practical Algorithms*. Publishing House of Electronics Industry.

- [2] Fu Qingxiang, Wang Xiaodong. *Algorithms and Data Structures*. Publishing House of Electronics Industry.
- [3] Yan Weimin, Wu Weimin. *Data Structures*, second edition. Tsinghua University Press.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*. Higher Education Press and The MIT Press.